National Higher School of Advanced Technologies

Academic year 2026–2027 | First-year Master's degree

Machine Learning – Practical Session 4

Overfitting, data leakage and model generalization

1. Context and Learning Outcomes

Building on the previous session, this practical session uses a realistic student-survey dataset to investigate data leakage, overfitting and model generalization.

By the end of the session, you will be able to:

- Inspect data for duplicates, missing values, rare categories and temporal patterns.
- Detect and measure data leakage caused by inappropriate splitting, duplicates and preprocessing.
- Diagnose overfitting by comparing training and testing errors.
- Improve generalization using regularization, feature selection and model complexity control.
- Select and evaluate the best model using validation data and an untouched test set.

2. Dataset and Task

File: `hybrid_student_performance_1200.csv` (1,200 survey responses, 36 columns, submitted between 15 March and 14 April 2026).

Every column is a self-reported survey answer. Most are categorical ranges (for example "1–2 hours" or "76% – 85%"), and between 2% and 5% of the answers are missing in several columns. The rows are **not** stored in chronological order. You will not use all 36 columns: the table below lists the ones needed in this session and how to encode them.

Column	Role and encoding
`student_id`	Unique student identifier (not a predictive feature).
`timestamp`	Survey submission date and time; defines the chronological order. If kept as a feature, convert it to `submission_day` = number of days since the first submission.
`cgpa_category`	Self-declared CGPA band, converted to the numeric **target** `final_cgpa` using the band midpoint: 5.0–6.9 → 5.95; 7.0–8.4 → 7.70; 8.5–9.4 → 8.95; 9.5–10.0 → 9.75.
`performance_risk_level`	Label: Low / Moderate / High Risk. Not a feature candidate: see Part A.
`study_hours_daily`	Renamed `study_hours`; midpoint in hours: Less than 1 hour → 0.5; 1–2 hours → 1.5; More than 2 hours → 2.5.
`sleep_hours`	Midpoint in hours: 4–5 hours → 4.5; 6–7 hours → 6.5; More than 8 hours → 8.5.
`attendance_percentage`	Renamed `attendance_rate`; midpoint in %: Less than 50% → 40; 50–65% → 57.5; 66–75% → 70.5; 76–85% → 80.5; Above

	85% → 92.5.
`study_consistency`	Ordinal 0–2: Rarely, Sometimes, Mostly consistent.

Column	Role and encoding
`tasks_on_time`, `assignments_on_time`	Ordinal 0–3: Rarely, Sometimes, Often, Always.
`revision_frequency`	Ordinal 0–3: Never, Rarely, Few times a week, Daily.
`career_goal_clarity`	Ordinal 0–2: Not clear, Somewhat clear, Very clear.
`daily_productivity`, `stress_level`, `age`	Already numeric (ratings from 1 to 5; age in years).
`cohort_code`	High-cardinality categorical feature **built by you**: `program_stream + " | " + year_class` (42 categories, some very rare).
`effort_index`	Feature **built by you** (used in Parts C and D): mean of `study_hours`/2.5, `attendance_rate`/100, `study_consistency`/2, `tasks_on_time`/3, `assignments_on_time`/3 and `revision_frequency`/3, ignoring missing answers within the row. The divisors are fixed by the encoding, so no statistic is learned from the data. It is needed because `study_hours` alone takes only three distinct values.

Prediction task: predict `final_cgpa` with regression models.

Evaluation metrics: RMSE (lower is better, same unit as the target, i.e. CGPA points) and R^2 (proportion of variance explained). Because the target is noisy, always report as a reference the RMSE of a baseline that predicts the training mean (`DummyRegressor`).

Ground rules for the whole session

- Use a fixed random seed (for example `random_state=42`) so results are reproducible. • Sort by `timestamp` once and reuse the **same chronological split** (first 80% for training, last 20% for testing) from Part B onward.
- Put every preprocessing step (missing-value imputation with the median, scaling, encoding) inside a scikit-learn `Pipeline`; never fit a transformer on test data.
- Use the test set **only once**, at the end, to report final results. Choose features and hyperparameters with validation data only (`TimeSeriesSplit` on the training set).

Part A. Data Inspection and Potential Leakage

Before training any model, investigate the dataset for characteristics that could affect generalization.

1. Load the CSV and examine its dimensions, data types, descriptive statistics and missing values (which columns, and what percentage of rows). Convert `timestamp` to a datetime.
2. Detect duplicate observations with `duplicated()` and report how many there are (with and without `student_id`, and ignoring `timestamp`). If there are none, say so: Part B.2 introduces duplicates in a controlled way.
3. Build `final_cgpa` and the encoded features of Section 2. Compute the correlation between each numerical feature and the target; comment on the strongest and weakest.
4. Compare `performance_risk_level` with `cgpa_category` (for example with `pd.crosstab`). Can this label be used as a predictive feature? Why or why not?
5. Study the distribution of `cohort_code`: how many categories have fewer than 10 observations?

6. Check whether the file is already in chronological order, sort it by `timestamp`, and plot `final_cgpa` over time (for example a 7-day rolling mean). Is there a trend or drift?

Expected output: a short list (3–5 bullet points) of risks found in the data and the leakage or overfitting problem each one may cause.

Part B. Impact of Data Leakage and Evaluation Strategy

The objective is to show how an inappropriate evaluation strategy produces misleading performance estimates. Remove `student_id` and `performance_risk_level`, and prepare the remaining predictive features: the eleven numeric and ordinal features of Section 2 (`study_hours`, `sleep_hours`, `attendance_rate`, `study_consistency`, `tasks_on_time`, `assignments_on_time`, `revision_frequency`, `career_goal_clarity`, `daily_productivity`, `stress_level`, `age`) plus `submission_day`. Set `cohort_code` aside until D.1. Impute missing values with the median inside the pipeline.

B.1 Compare splitting strategies

Train a `DecisionTreeRegressor(random_state=42)` with two strategies:

- **Random split:** 80% training / 20% testing.
- **Chronological split:** first 80% for training, last 20% for testing.

Report training and testing RMSE for both (and the baseline RMSE), and explain the difference.

B.2 Investigate duplicate leakage

The CSV contains no exact duplicates, so simulate a classic survey problem: students who submit the form twice. Build an augmented dataset in which 15% of the rows, drawn at random (`random_state=42`), are duplicated. Each copy keeps the same answers and `student_id` but receives a new random timestamp within the survey period (recompute `submission_day` afterwards). With the same model and the chronological split, compare:

- Splitting **before** removing duplicates.
- Removing duplicates **before** splitting (identify duplicates on all columns except `timestamp`). Explain how a duplicated row can end up in both the training and the test set, and what this does to the test score.

B.3 Investigate preprocessing leakage

Use `KNeighborsRegressor(n_neighbors=10)` for this comparison (a decision tree is insensitive to feature scaling and would show no difference). Compare two ways of standardizing features:

- Fit `StandardScaler` on the **entire dataset** before splitting.
- Fit `StandardScaler` on the **training data only**, inside a `Pipeline`.

Compare testing RMSE and explain why preprocessing statistics (scaling, but also the median used for imputation) must be learned from training data only. The numerical gap may be small here; explain why the principle still matters.

Expected output: one summary table (strategy, training RMSE, testing RMSE) and a short written explanation for each of B.1, B.2 and B.3.

Part C. Understanding and Diagnosing Overfitting

Use the chronological training and testing sets established in Part B.

C.1 Effect of model complexity

Train `DecisionTreeRegressor` models with:

```
max_depth = [1, 2, 3, 5, 10, 20, None]
```

- Compute the training and testing RMSE for each depth.
- Plot both errors against tree depth.
- Compute the generalization gap for each depth:

$$Generalization\ gap = RMSE_{test} - RMSE_{train}$$

Identify the depth at which the model starts to overfit.

C.2 Effect of feature complexity

Use polynomial features of `effort_index` (Section 2) with degrees:

```
degree = [1, 2, 3, 5, 9, 15]
```

Build a `Pipeline` with `PolynomialFeatures` → `StandardScaler` → `LinearRegression`, and evaluate each degree with `TimeSeriesSplit` (5 folds) on the training set only. Plot cross-validation RMSE against the polynomial degree.

Expected output: two plots (RMSE vs. depth; CV RMSE vs. degree) with a one-paragraph interpretation of each. In C.2, comment on the size of the effect compared with C.1 and explain why.

Part D. Improving Model Generalization

Apply appropriate techniques to improve the models studied in Part C. In D.1 to D.3, compare options with validation RMSE (`TimeSeriesSplit`, 5 folds, on the training set); the test set is used only in D.4.

D.1 Handling a high-cardinality categorical feature

Compare three treatments of `cohort_code` (42 categories) using a `DecisionTreeRegressor(max_depth=5, random_state=42)` and the Part B features:

Approach	Description
Rare-category grouping	Group categories with fewer than 10 **training** observations into "other" (hint: `OneHotEncoder(min_frequency=10, handle_unknown="infrequent_if_exist")`).
Target encoding	Encode each category with its target mean, computed **out-of-fold** to avoid leakage (hint: `TargetEncoder(cv=5, shuffle=False)`).

Feature removal	Exclude `cohort_code` from the model.

Which treatment is best, and what does the result tell you about how informative `cohort_code` is?

D.2 Feature selection

`submission_day` increases with time, so in a chronological split every test observation lies beyond the range seen during training. Using validation RMSE, compare Linear Regression, a Decision Tree (`max_depth=5`) and `GradientBoostingRegressor(n_estimators=60, learning_rate=0.05, max_depth=3)` **with and without** `submission_day`. Decide whether to keep it for the rest of the session and justify your choice. Optionally, test whether removing the features with a correlation close to zero (Part A) changes validation RMSE.

D.3 Applying regularization

Using polynomial features of degree 5 of `effort_index`, compare Linear Regression, Ridge and Lasso. Test the regularization strengths:

$$\alpha \in \{0.001,\ 0.01,\ 0.1,\ 1,\ 10,\ 100\}$$

Use a `Pipeline` (`PolynomialFeatures` → `StandardScaler` → model) and `TimeSeriesSplit` to select the best α for each model. Comment on how Ridge and Lasso change the coefficients compared with the unregularized model (for example, how many Lasso coefficients are exactly zero?).

D.4 Comparing the final models

Compare the best configuration of each model on the **same chronological test set**, using the same feature matrix for all models (Part B features after the choices made in D.1 and D.2):

- **Linear Regression:** no penalty.
- **Ridge or Lasso:** α selected with `TimeSeriesSplit` on this feature matrix (no polynomial expansion).
- **Decision Tree:** `max_depth` selected with `TimeSeriesSplit` among the values of C.1. • **Gradient Boosting with early stopping:** keep the last 15% of the training set (chronologically) as a validation block, use `staged_predict` to find the number of trees that minimizes validation RMSE, then refit with that number on the full training set. (The built-in early stopping options of scikit-learn use a random validation split, which breaks the chronological order.)

Complete the table:

Model	Training RMSE	Testing RMSE	Testing R^2
Baseline (training mean)			
Linear Regression			
Ridge / Lasso			
Decision Tree			
Gradient Boosting			

Final Discussion

Answer in a few sentences each, supporting your claims with your results:

1. Which model achieves the best generalization?
2. Which model shows the largest generalization gap?
3. Which techniques were most effective in reducing overfitting?
4. Why should model selection rely on validation performance rather than training performance?
5. Why must the final test set remain untouched during model selection?
6. The best R^2 is modest. What does this tell you about the information contained in the survey features, and why is a high score not always a sign of a good evaluation?

Deliverable

Submit a single well-organized, runnable notebook or script (**code**) containing results, plots and short written answers. Place `hybrid_student_performance_1200.csv` in the same folder and load it with a relative path.